When we move to the implementation, however, constant factors matter. They matter so much that, in some edge cases, algorithms that would be better according to the running time analysis actually are slower than simpler algorithm with worse running time, at least for any practical input (for instance if you compare `2n` and `n*100100000`, then for the constant factor not to matter anymore, the input should be huge).

A prominent example of this behavior is given by Fibonacci heap[21]: in theory they provide amortized constant time for some crucial operations like delete or priority update, but in practice they are both complicated to implement and slow for any viable input size.

The constant factors, in general, are due to several different reasons.

These includes:

- Lag for reading/writing to memory (scattered vs localized readings).
- The cost of maintaining counters or to iterate loops.
- The cost of recursion.
- Nitty/gritty coding details that in the asymptotic analysis are abstracted away. (For instance, as we have seen, static vs dynamic arrays).

So, at this point it should be clear that in any implementation we should strive to keep this constant multiplicators as small as possible.

Consider this formula: